

# Anchor Parsing

May 2026

**Abstract** We present a parser that given a context free grammar  $G = (N, \Sigma, P, S)$  and a string  $\beta$  such that  $\exists x, y \in \Sigma^* : x\beta y \in L(G)$ , enumerates all possible parse trees that can cover  $\beta$  with the missing left and right context, if any. We can use this to extract parse trees from incomplete code, such as fragments found in git patches.

## 1 Introduction

Git patches only record the added and removed lines in a codebase, presenting the code as fragments with minimal surrounding context. When studying how code changes across a series of patches, there is a need to understand what the changed code is syntactically: a declaration, a function call, a statement, etc.

```
int size, rc = 0;

while (n > 0) {
- size = zpci_get_max_write_size((u64 __force)
src,
- (u64) dst, n,
- ZPCI_MAX_READ_SIZE);
+ size = zpci_get_max_io_size((u64 __force) src,
+ (u64) dst, n,
+ ZPCI_MAX_READ_SIZE);
rc = zpci_read_single(dst, src, size);
if (rc)
break;
```

*A fragment from a Linux kernel patch.*

```
int size, rc = 0;

while (n > 0) {
size = zpci_get_max_io_size((u64 __force) src,
(u64) dst, n,
ZPCI_MAX_READ_SIZE);
rc = zpci_read_single(dst, src, size);
if (rc)
break;
```

*Code to be parsed (highlighted) with surrounding context.*

Parsing this input is complicated, since standard parsers assume a well-formed, complete input. When presented with a fragment they fail at the first token that cannot extend a valid parse due to the missing surrounding context. Parsers with error recovery strategies (tree-sitter and similar tools) salvage a single best parse but suppress ambiguity that might exist.

To resolve these issues, we change the basic assumptions.

1. We assume that the input string belongs to the language and has a valid derivation
2. We only need the smallest non terminals that can cover the entire input.

Given a context-free grammar  $G$  and a token sequence  $\beta$ , our algorithm returns the covering set  $\mathcal{C}(\beta) \subseteq N$  of nonterminals under which  $\beta$  appears as a substring, together with parse trees rooted at nonterminals in  $\mathcal{C}(\beta)$  and descriptions of the missing context. This allows us to compare two fragments with the same covering set directly as instances of the same grammatical construct.

In this paper we describe an adaptation of the h-cover framework of [1] to the fragment parsing problem, returning  $\mathcal{C}(\beta)$  with parse trees and context descriptions for any context-free grammar. We evaluate it on real fragments drawn from Linux kernel patches.

## 2 Problem

Let  $G = (N, \Sigma, P, S)$  be a context-free grammar and  $\beta \in \Sigma^+$  a token sequence. A nonterminal  $A \in N$  covers  $\beta$  if there exist  $\alpha, \gamma \in \Sigma^*$  such that

$$A \xRightarrow{*} \alpha\beta\gamma$$

The *covering set* is

$$\mathcal{C}(\beta) = \{A \in N \mid \exists \alpha, \gamma \in \Sigma^* : A \xRightarrow{*} \alpha\beta\gamma\}$$

The algorithm computes, for each  $A \in \mathcal{C}(\beta)$ , a set of *gap parse trees* rooted at  $A$  spanning  $\beta$ . A gap parse tree is a parse tree over  $\beta$  whose leaves are either tokens from  $\beta$  or *virtual nodes*  $\langle X \rangle$  ( $X \in N \cup \Sigma$ ). Virtual nodes mark constituents present in the derivation of  $A$  but absent from  $\beta$ . Those appearing before the first token form the left gap  $L$ ; those after the last token form the right gap  $R$ .

**Example.** Let  $G_{cl}$  be the grammar [1]:

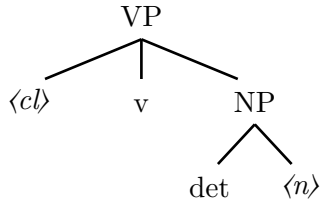
$$\begin{aligned} S &\rightarrow NP \ VP \quad (\text{head: VP}) \\ VP &\rightarrow cl \ v \ NP \quad (\text{head: v}) \\ NP &\rightarrow det \ n \quad (\text{head: det}) \end{aligned}$$

For the fragment

$$\beta = v \ det$$

VP derives  $cl \cdot \beta \cdot n$ , placing it in the covering set. S derives  $NP \ cl \cdot \beta \cdot n$  similarly, giving  $\mathcal{C}(\beta) = \{VP, S\}$ .

The gap parse tree for VP (left gap  $\langle cl \rangle$ , right gap  $\langle n \rangle$ ) is:



$\langle cl \rangle$  is the missing left sibling and  $\langle n \rangle$  the missing right terminal, giving  $L_{VP} = (\langle cl \rangle)$  and  $R_{VP} = (\langle n \rangle)$ . Boundary seeding promotes VP into S, prepending  $\langle NP \rangle$ , so  $L_S = (\langle NP \rangle, \langle cl \rangle)$  and  $R_S = (\langle n \rangle)$ .

### 3 Overview of the algorithm

Given a CFG  $G$  and an input  $\beta$ , each production in  $G$  designates one symbol as the *head* [1]. Using these heads, we precompute an intermediate grammar called the *h-cover*  $\mathcal{H}(G)$ . In  $\mathcal{H}(G)$ , the productions expand outwards from the selected head, and the binary nature of the productions means that every combination step merges exactly two spans. This allows us to use tabular methods to enumerate all valid derivations over every span of  $\beta$  in cubic time.

Every token in  $\beta$  is a potential starting point for a derivation. Initially, each token initiates a chain of upward projections through the grammar, each step applying a head cover production until no further projections apply. From there, analysis expands bidirectionally, combining with constituents to the left and right in any order until no new derivations can be found.

While this suffices for a complete input, a fragment may be missing its left or right context. The first or last token of  $\beta$  may need a left or right sibling that is absent from the input. To handle this, we seed the boundary cells  $T[0, 1]$  and  $T[n - 1, n]$  with all items reachable by inferring the missing left or right constituent from the grammar, a step we call *boundary seeding*. Each inferred constituent is recorded as a virtual node in the resulting parse tree.

If  $T[0, n]$  is empty after the main agenda, for each prefix span  $T[0, k]$ , the agenda processes items there with a virtual left sibling, producing new items that can reach  $T[0, n]$ . If  $T[0, n]$  is still empty after this, then we do the same from suffix spans  $T[k, n]$ . The former is called L-Reduce, and the latter R-Reduce. A final agenda pass then runs on any items newly arrived in  $T[0, n]$ .

The items in  $T[0, n]$  at the end span the full fragment. Each complete item there identifies a

covering nonterminal, and the parse tree reconstructed from it carries the virtual nodes as leaves, directly giving  $L_A$  and  $R_A$ .

## 4 Algorithm

### 4.1 Recognition Table

Let  $\beta = w_1 w_2 \dots w_n$  be the input fragment. The recognition table  $T$  is an  $(n + 1) \times (n + 1)$  array where  $T[i, j]$  holds items representing partial or complete derivations over  $w_{\{i+1\}} \dots w_j$ .

Two item types are used. A **complete item** for nonterminal  $A$  asserts that  $A$  derives  $w_{\{i+1\}} \dots w_j$ . A **partial item** for production  $r$  asserts that a head-adjacent slice of the right-hand side has been assembled over  $w_{\{i+1\}} \dots w_j$ , and the remaining symbols to the left and right are yet to be combined.

### 4.2 H-Cover

The  $h$ -cover  $\mathcal{H}(G)$  is computed once from the grammar and reused across all inputs. For each production  $D \rightarrow Z_1 \dots Z_\pi$  with head  $Z_\tau$ , the cover records:

- **Projections** — a complete item for  $D$  can be projected once the head partial item is complete.
- **Left expansions** — a partial item can be extended leftward by combining with symbol  $Z_{\tau-1}, Z_{\tau-2}, \dots$  found elsewhere in the table.
- **Right expansions** — symmetrically, the partial item is extended rightward.

### 4.3 Agenda

Recognition fills  $T$  via an agenda. When item  $a$  is added to  $T[i, j]$  for the first time it is enqueued. Processing  $a$  applies four rules:

1. **Project:** add the complete item projected from  $a$ .
2. **Left-expand:** find all  $i' \leq i$  such that the required left symbol is in  $T[i', i]$ ; add the extended partial item to  $T[i', j]$ .

3. **Right-expand:** find all  $j' \geq j$  such that the required right symbol is in  $T[j, j']$ ; add the extended partial item to  $T[i, j']$ .
4. **Reverse:**  $a$  may be the missing child in an already-triggered expansion. Scan for matching triggers and produce the resulting item.

The agenda terminates because each item is enqueued at most once and  $T$  is finite.

### 4.4 Boundary Seeding

The agenda alone fills  $T[0, n]$  only when the fragment has sufficient left context starting from  $w_1$ . For fragments whose left or right boundary is missing, two additional passes are needed.

**L-Reduce** processes prefix spans  $T[0, k]$  for increasing  $k$ . If an item at  $T[0, k - 1]$  is a right child of some expansion whose left child is absent from the input, a virtual left sibling is injected and the agenda is re-run. The virtual sibling represents the missing left context.

**R-Reduce** (applied only if  $T[0, n]$  is empty after L-Reduce) does the same in the other direction, injecting virtual right siblings into suffix spans  $T[k, n]$ .

A final closure step runs on  $T[0, n]$  directly, combining prefix and suffix derivations that meet in the middle.

### 4.5 Root Extraction

Items in  $T[0, n]$  span the entire fragment, possibly with virtual gaps. For each complete item  $I_A \in T[0, n]$ , we climb the cover productions to find the covering nonterminal  $A$  and collect the virtual gap descriptors accumulated during boundary seeding. This yields the root candidates:  $(A, T_A, \alpha, \gamma)$ , sorted by gap count.

## 5 Implementation

The implementation is in OCaml and separates grammar preparation from recognition.

**Prepare( $G$ )**

1. For each production and head position: add projections, left/right expansions to  $\mathcal{H}$
2. Compute nullable set; add epsilon-projection rules
3. Return  $\mathcal{H}(G)$

**Recognize( $\mathcal{H}, \beta$ )**

1. Initialise  $T[i, j] = \emptyset$  for all  $i, j$
2. Seed terminals into  $T[k - 1, k]$  from projections
3. Run agenda
4. L-Reduce: inject virtual left gaps; re-run agenda
5. If  $T[0, n] = \emptyset$ : R-Reduce symmetrically
6. Final closure on  $T[0, n]$
7. Return  $T$

**Query( $T, n$ )**

1. For each  $I_A \in T[0, n]$ : climb productions; collect gap descriptors
2. Return root candidates sorted by gap count

Grammar preparation is performed once per grammar; the resulting  $\mathcal{H}(G)$  is reused across all fragments. This is significant in the patch analysis setting, where thousands of fragments from the same language are processed in sequence.

## 6 Evaluation

We evaluate on three grammars of increasing complexity. Grammar preparation runs once per grammar; recognition and reconstruction run per fragment.

**GCL grammar.** Three productions, 4 terminals. For the fragment  $\beta = v \text{ det}$  the system returns one root: VP with 2 gaps ( $L = \langle \text{cl} \rangle$ ,  $R = \langle \text{n} \rangle$ ) and a single parse tree. S is inferred but removed by the subtree-dominance filter, since S’s best tree contains VP as a direct child — retaining both would be redundant. For the complete input “det n cl v det n” the system returns S with 0 gaps and a single tree.

**Lisp grammar.** An S-expression grammar read from an ANTLR4 file, with alternation

and Kleene operators desugared by the grammar pipeline. Single-atom inputs are covered by `s_expression` with 0 gaps. Dotted pairs such as `(ATOM . ATOM)` are likewise covered with 0 gaps.

**C grammar.** A full C grammar in ANTLR4 format, desugared to several hundred productions. Grammar preparation completes in under one second and is shared across all fragments. For short fragments of 2–10 tokens, recognition produces tens to hundreds of table items. Multiple root candidates appear naturally: a bare sequence of tokens may be a valid expression, a statement argument, a declarator, or several simultaneously. The subtree-dominance filter reduces the displayed roots to non-redundant candidates. Tree reconstruction is lazy and capped at 5 trees per root to avoid cartesian-product blowup in the derivation forest; this suffices to identify the syntactic category and gap description for each root.

## 7 Related Work

**Substring recognition.** Rekers and Koorn [2] give an  $O(n^3)$  algorithm for deciding whether  $\beta$  is a substring of some sentence in  $L(G)$ , and produce parse trees. Their trees are for the complete sentence  $\alpha\beta\gamma$  rooted at  $S$ , not for  $\beta$  alone rooted at covering nonterminals. The covering set  $\mathcal{C}(\beta)$  is not computed.

Osorio and Navarro [3] give the closest prior formulation: using a CYK variant they compute  $\{A \in N \mid \beta \in L^{\text{infix}(A)}\}$  explicitly. No parse trees are produced, no context description is given, and inputs that are not clean infixes of any sentence are not handled.

**Gap parsing.** Lang [4] parses strings with explicit gap markers (? for one unknown word, \* for a sequence). The parser labels gap positions with covering nonterminals. Our setting differs: we have no markers and no knowledge of gap

positions — the entire surrounding context is unknown.

**Bidirectional parsing.** Satta and Stock [1] develop the h-cover framework for bidirectional tabular recognition of complete input. We adopt the framework but change the goal: instead of recognising membership in  $L(G)$ , we classify a bare fragment by computing  $\mathcal{C}(\beta)$  with parse trees and context descriptions. Partial items, intermediate in the original algorithm, become the primary output of ours.

**Error recovery.** Tree-sitter and similar tools find a single best repair when parsing fails. This is appropriate for editors but not for patch analysis, where multiple grammatical classifications may simultaneously hold for the same fragment.

## 8 Conclusions

We have defined the fragment parsing problem and described an algorithm for solving it: given any context-free grammar and a bare token sequence, compute the covering set  $\mathcal{C}(\beta)$  of nonterminals whose infix language contains  $\beta$ , together with parse trees and context descriptions for each. The algorithm adapts the h-cover framework of Satta and Stock to operate without a complete input or a privileged start symbol. We evaluate it on C fragments from Linux kernel patches.

**Limitations.** Tree reconstruction is expensive for long fragments against large grammars; lazy reconstruction is future work. The algorithm assumes the fragment has a valid derivation — genuinely malformed inputs are not currently flagged as such.

**Future work.** Probabilistic ranking of root candidates; application to additional languages via existing ANTLR grammars; a study of syntactic change patterns in Linux kernel patches using  $\mathcal{C}(\beta)$  as the classification signal.

## Bibliography

- [1] G. Satta and O. Stock, “BiDirectional Context-Free Grammar Parsing for Natural Language Processing,” *Artificial Intelligence*, vol. 69, pp. 123–164, 1994.
- [2] J. Rekers and W. Koorn, “Substring Parsing for Arbitrary Context-Free Grammars,” in *Proceedings of the Second International Workshop on Parsing Technologies (IWPT)*, 1991, pp. 218–224.
- [3] M. Osorio and J. A. Navarro, “Decision Problem of Substrings in Context Free Languages,” in *Proceedings of CIC 2001*, 2001.
- [4] B. Lang, “Parsing Incomplete Sentences,” in *Proceedings of the 12th International Conference on Computational Linguistics (COLING)*, 1988, pp. 365–371.